Harvest Bistro Project Final Report

1. Introduction

Restaurants often struggle to manage reservations, tables, and menu updates efficiently when using manual processes or disconnected systems. These challenges can lead to issues such as double bookings, poor table allocation, and limited visibility into customer trends and popular menu items.

The goal of this project was to design and implement a web-based Restaurant Reservation and Menu Management System for a fictional restaurant, Harvest Bistro. The system allows customers to browse menu items, make reservations, and manage their bookings, while enabling staff and administrators to oversee restaurant operations, update menu data, and analyze reservation trends.

The system supports three main user roles:

- Customers - browse menu, make and cancel reservations
- Staff - manage reservations, assign tables, update menu items
- Admin - access reporting dashboards, analyze trends, manage staff

This project demonstrates a real-world application of database design by integrating a structured relational database with a functional web interface and backend system.

2. ER Diagram (Conceptual Design)

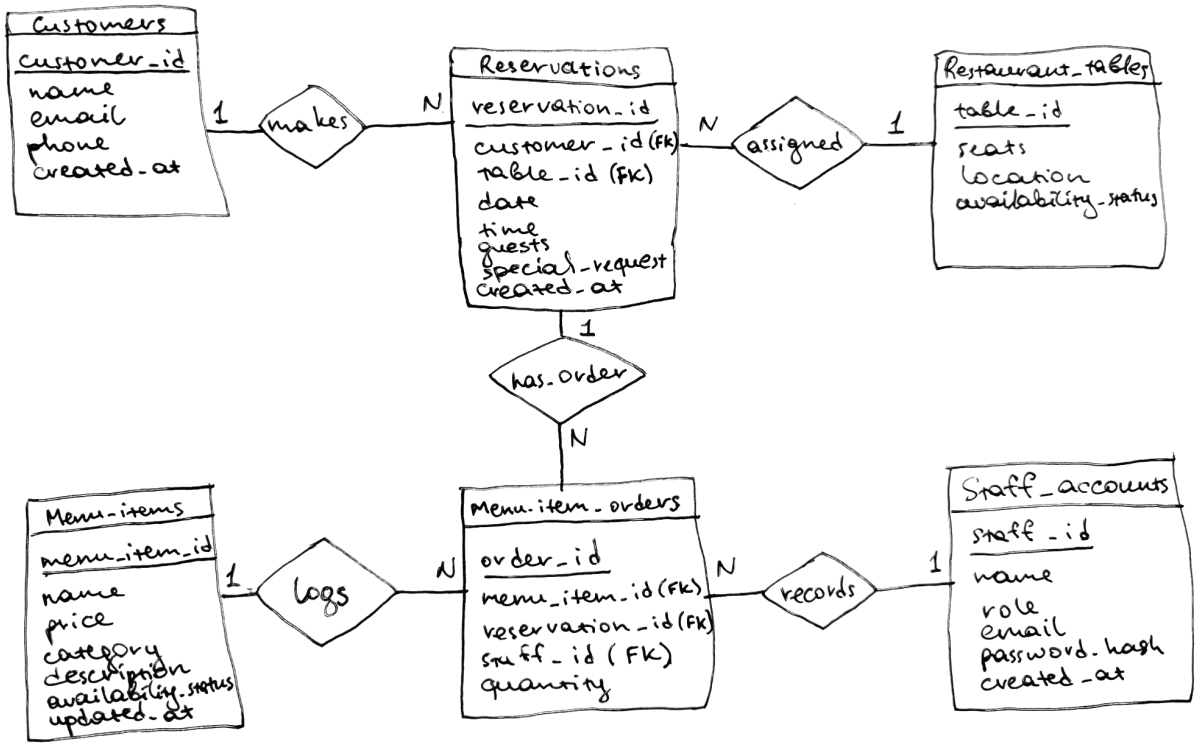
Based on the final SQL schema, the system consists of six main entities:

- Customers
- Reservations
- Restaurant_Tables
- Menu_Items
- Staff_Accounts
- Menu_Item_Orders

Relationships

- A **Customer -> Reservations** relationship (1:M)
  One customer can create multiple reservations

- A **Reservation -> Restaurant_Table** relationship (M:1)
  Each reservation is assigned to one table (optional)
- A **Reservation -> Menu_Item_Orders** relationship (1:M)
  A reservation can have multiple ordered items
- A **Menu_Item -> Menu_Item_Orders** relationship (1:M)
  A menu item can appear in multiple orders
- A **Staff -> Menu_Item_Orders** relationship (1:M)
  Staff members record order



3. Database Design

The database was implemented using MySQL and follows relational database design principles with clearly defined primary and foreign keys.

Core Tables

- **customers**
  - Stores customer details (name, email, phone)
  - Each Customer is assigned a unique customer_id which acts as their primary key
- **restaurant_tables**
  - Stores table information such as seating capacity and location

- **menu_items**
    - Contains all menu offerings, including category, price, and availability
- **staff_accounts**
    - Stores login credentials and roles (staff or admin)
- **reservations**
    - Links customers to reservations
    - Includes date, time, number of guests, and status
    - Foreign keys ensure data integrity
- **menu_item_orders**
    - Tracks items ordered during a reservation
    - Connects menu items, reservations, and staff

Normalization

The database is normalized to Third Normal Form (3NF):

- No repeating groups (1NF)
- All non-key attributes depend on the primary key (2NF)
- No transitive dependencies (3NF)

For example:

- Customer data is separated from reservations
- Menu items are independent from orders
- Orders act as a junction table connecting multiple entities

This ensures reduced redundancy and improved data consistency.

4. Implementation Details (System + Code)

The system was implemented using PHP (backend), MySQL (`database`), HTML/CSS (frontend), and JavaScript (interactivity).

Database Connection

The system uses a centralized database connection via PDO:

```
$pdo = new PDO(

'mysql:host=127.0.0.1;port=3306;dbname=restaurant_db;charset=utf8mb4',
  'root',
```

```
  ''
);
```

This ensures:

- secure database access
- reusable connection across all files
- consistent error handling

Reservation System

Reservations are handled in `reserve.php`.

Key features:

- Input validation
- Existing customer detection
- Transaction-based insertion

```
$customerStmt = $pdo->prepare(
  'SELECT customer_id FROM customers WHERE email = :email'
);
```

If the customer exists, their data is updated; otherwise, a new record is created. Then a reservation is inserted into the database.

Reservation Cancellation

Handled in `cancel_reservation.php`:

```
UPDATE reservations r
JOIN customers c ON c.customer_id = r.customer_id
SET r.status = "cancelled"
WHERE r.reservation_id = :reservation_id
AND c.email = :email
```

This ensures:

- only the correct user can cancel

- validation through email matching

The customer's information will stay in the system after the cancellation, this is in case the reservation was cancelled by mistake.

Authentication System

Login is handled using hashed passwords:

```
if ($user && password_verify($password, $user['password_hash']))
```

- Sessions store logged-in users
- Role-based redirects:
  - Admin -> admin dashboard
  - Staff -> staff dashboard
- Passwords are not hardcoded into the database

Staff Operations

Staff can:

- update reservation status
- assign tables
- edit or add menu items

Example:

```
UPDATE reservations
SET status = :status, table_id = :table_id
WHERE reservation_id = :reservation_id
```

Admin Dashboard (Analytics)

Admin features include advanced SQL queries such as:

Reservation statistics

```
SELECT COUNT(*) AS total_reservations,
SUM(status = "confirmed") AS confirmed_count
FROM reservations
```

Peak hours

```
SELECT DATE_FORMAT(reservation_time, "%H:00"), COUNT(*)
FROM reservations
GROUP BY reservation_hour
```

Popular menu items

```
SELECT mi.name, SUM(mio.quantity)
FROM menu_item_orders mio
JOIN menu_items mi ON mi.menu_item_id = mio.menu_item_id
GROUP BY mi.menu_item_id
```

These queries provide meaningful business insights for staff and admin. Data such as popular menu items provide insight on what time of day the dishes are most popular. This can allow the servers and staff to better prepare for their shifts.

Frontend Interactivity

JavaScript enhances usability:

```
fetch("reserve.php", {
  method: "POST",
  headers: { "X-Requested-With": "XMLHttpRequest" },
  body: new FormData(form)
});
```

This allows:

- real-time reservation submission
- dynamic UI updates
- filtering menu categories

5. Challenges

Several challenges were encountered during development:

- **Frontend and backend integration**
  Connecting PHP logic with UI elements required careful coordination. It was vital each teammate knew how to properly load and view the website to ensure there were not format or compatibility errors.
- **Environment setup (PHP + MySQL)**
  Ensuring all team members could run the project locally was difficult. Instead of using XXAMP, we each installed mysql and php through HomeBrew.
- **Branching and collaboration**
  Managing multiple contributors working on different parts of the system required consistent version control.
- **Database synchronization**
  Keeping schema and application logic aligned across features. Some values are hard coded into the schema, all frontend operations are stored locally on each individual device, until “mysql -u root -p < schema.sql” is run in the terminal. This command re-runs the database on MySql which drops the tables if already created.

Despite these challenges, the team successfully implemented all major features.

6. Future Work

While the system is fully functional, several improvements can enhance its capabilities:

- Real-time table availability tracking
- Expand the Menu by adding more dishes
- Online payment integration
- Customer accounts with booking history
- Advanced analytics (revenue, customer trends)
- Mobile responsiveness and UI refinement
- Notification system (email confirmations, reminders)

These additions would make the system closer to a production-level restaurant management platform.

Conclusion

The Harvest Bistro system successfully demonstrates how a relational database can support a real-world application. By combining structured database design with a dynamic web interface, the project enables efficient reservation management, menu tracking, and operational analytics.

The project highlights the importance of:

- proper database design

- role-based system architecture
- integration between frontend and backend

Overall, this system provides a strong foundation for scalable restaurant management solutions.